

Technology Stack Reference Guide

Overview

This guide provides complete setup instructions for all technology stacks and tools used in the AI Readiness Training Program. Complete the setup for **your chosen backend stack** (Python, .Net, or Java) plus all the shared tools (Google AI Studio, LangSmith, ChromaDB, GitHub Copilot).

Estimated setup time: 2–3 hours (Day 1 of Phase 1)

Section 1: Python Stack Setup

1.1 Python Installation

- Download Python **3.11+** from python.org
- Verify: `python --version` should show 3.11.x or higher
- Install pip: usually bundled with Python 3.11+

1.2 Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (Mac/Linux)
source venv/bin/activate

# Verify activation – you should see (venv) in your prompt
```

1.3 Required Packages

Create a `requirements.txt` in your project root:

```
# Web Framework
fastapi==0.111.0
uvicorn[standard]==0.29.0

# Database
sqlalchemy==2.0.30
alembic==1.13.1

# Validation
pydantic==2.7.1
pydantic-settings==2.3.0
```

```
python-multipart==0.0.9

# Authentication
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# Logging
structlog==24.1.0

# OpenTelemetry
opentelemetry-sdk==1.25.0
opentelemetry-instrumentation-fastapi==0.46b0
opentelemetry-exporter-otlp==1.25.0

# LangChain (Phases 2-4)
langchain==0.2.6
langchain-google-genai==1.0.6
langchain-community==0.2.6
langchain-chroma==0.1.2
chromadb==0.5.3
langsmith==0.1.77

# MCP (Phase 4)
fastmcp==0.4.1

# UI (Phases 2-4)
streamlit==1.36.0
gradio==4.37.2

# LangGraph (Phase 5)
langgraph==0.1.17

# Testing
pytest==8.2.2
pytest-asyncio==0.23.7
httpx==0.27.0
pytest-cov==5.0.0

# Utilities
python-dotenv==1.0.1
requests==2.32.3
```

Install all packages:

```
pip install -r requirements.txt
```

1.4 Environment Variables (.env file)

Create a **.env** file in your project root (never commit this to git):

```
# Application
APP_NAME=your-poc-name
APP_ENV=development
DEBUG=true
SECRET_KEY=your-secret-key-min-32-chars-long

# Database
DATABASE_URL=sqlite:///./app.db

# Google AI Studio
GOOGLE_API_KEY=your-google-ai-studio-api-key

# LangSmith
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your-langsmith-api-key
LANGCHAIN_PROJECT=AI-Readiness-POC-XX-P2

# OpenTelemetry
OTEL_SERVICE_NAME=poc-xx-phase-x
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317

# Associate ID (for logging)
ASSOCIATE_ID=your-name-or-id
POC_ID=POC-01
```

1.5 VS Code Extensions (Recommended)

Install these VS Code extensions:

- **Python** (ms-python.python)
- **Pylance** (ms-python.vscode-pylance)
- **GitHub Copilot** (GitHub.copilot)
- **GitHub Copilot Chat** (GitHub.copilot-chat)
- **SQLite Viewer** (qwtel.sqlite-viewer)
- **REST Client** (humao.rest-client)
- **Thunder Client** (rangav.vscode-thunder-client)

Section 2: .Net C# Stack Setup

2.1 .NET SDK Installation

- Download **.NET 8 SDK** from dotnet.microsoft.com
- Verify: `dotnet --version` should show 8.x.x

2.2 Project Creation

```
# Create solution
dotnet new sln -n YourPocName
```

```
# Create Web API project
dotnet new webapi -n YourPocName.Api -controllers

# Create test project
dotnet new xunit -n YourPocName.Tests

# Add projects to solution
dotnet sln add YourPocName.Api
dotnet sln add YourPocName.Tests

# Add test project reference
cd YourPocName.Tests
dotnet add reference ../YourPocName.Api/YourPocName.Api.csproj
```

2.3 NuGet Packages

```
cd YourPocName.Api

# Entity Framework Core + SQLite
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design

# Authentication
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

# Logging
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Formatting.Compact

# OpenTelemetry
dotnet add package OpenTelemetry.Extensions.Hosting
dotnet add package OpenTelemetry.Instrumentation.AspNetCore
dotnet add package OpenTelemetry.Exporter.Console

cd ../YourPocName.Tests
dotnet add package Microsoft.AspNetCore.Mvc.Testing
dotnet add package Moq
```

2.4 appsettings.json Template

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  },
  "Jwt": {
    "Key": "your-secret-key-min-32-chars",
    "Issuer": "your-poc-api",
```

```
"Audience": "your-poc-client",
"ExpiryMinutes": 60
},
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},
"GoogleAI": {
  "ApiKey": "your-google-api-key"
},
"LangSmith": {
  "ApiKey": "your-langsmith-api-key",
  "Project": "AI-Readiness-POC-XX-P2"
}
}
```

2.5 Serilog Setup (Program.cs)

```
using Serilog;
using Serilog.Formatting.Compact;

Log.Logger = new LoggerConfiguration()
    .WriteTo.Console(new CompactJsonFormatter())
    .Enrich.WithProperty("PocId", "POC-01")
    .Enrich.WithProperty("Phase", 1)
    .CreateLogger();

builder.Host.UseSerilog();
```

Section 3: Java Stack Setup

3.1 JDK Installation

- Download **JDK 21** (LTS) from adoptium.net
- Verify: `java --version` should show 21.x
- Verify Maven: `mvn --version`

3.2 Spring Boot Project Creation

Use Spring Initializr (start.spring.io):

- Project: Maven
- Language: Java
- Spring Boot: 3.3.x
- Java: 21
- Dependencies: Spring Web, Spring Data JPA, Spring Security, Validation, Lombok

Or via CLI:

```
# Using Spring CLI (if installed)
spring init --dependencies=web,data-jpa,security,validation,lombok --java-
version=21 your-poc-name
```

3.3 Maven Dependencies (pom.xml)

```
<dependencies>
  <!-- Spring Boot Starters -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- SQLite -->
  <dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.46.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-community-dialects</artifactId>
  </dependency>

  <!-- JWT -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
  </dependency>

  <!-- Lombok -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
```

```
</dependency>

<!-- OpenTelemetry -->
<dependency>
  <groupId>io.opentelemetry.instrumentation</groupId>
  <artifactId>opentelemetry-spring-boot-starter</artifactId>
  <version>2.5.0</version>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>
```

3.4 application.properties Template

```
# Application
spring.application.name=your-poc-name
server.port=8080

# SQLite
spring.datasource.url=jdbc:sqlite:app.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update

# JWT
app.jwt.secret=your-secret-key-min-32-chars
app.jwt.expiration=86400000

# Logging
logging.level.root=INFO
logging.level.com.yourpackage=DEBUG
logging.pattern.console=%d{ISO8601} [%thread] %-5level %logger{36} - %msg%n

# OpenTelemetry
otel.service.name=poc-xx-phase-x
otel.traces.exporter=otlp

# Google AI (Phase 2+)
google.ai.api-key=${GOOGLE_API_KEY}

# LangSmith (Phase 2+)
langsmith.api-key=${LANGCHAIN_API_KEY}
langsmith.project=AI-Readiness-POC-XX-P2
```

Section 4: Google AI Studio Setup

4.1 Getting Your Free API Key

1. Go to aistudio.google.com
2. Sign in with your personal Google account
3. Click **"Get API Key"** in the top navigation
4. Click **"Create API Key"**
5. Copy the key — it starts with **AIza...**
6. Add to your `.env` file: `GOOGLE_API_KEY=AIza...`

4.2 Free Tier Limits (Gemini 2.0 Flash)

Metric	Free Tier Limit
Requests per minute	15 RPM
Requests per day	1,500 RPD
Input tokens per minute	1,000,000 TPM
Output tokens per minute	32,000 TPM
Context window	1,000,000 tokens

Tip: For POC work with a single associate, free tier limits are more than sufficient. If you hit rate limits, add `time.sleep(4)` between LLM calls in batch operations.

4.3 Testing the Connection

```
# test_gemini.py
from langchain_google_genai import ChatGoogleGenerativeAI
import os
from dotenv import load_dotenv

load_dotenv()

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key=os.getenv("GOOGLE_API_KEY"),
    temperature=0.1
)

response = llm.invoke("Hello! What is 2+2?")
print(response.content) # Should print: 4 (or "The answer is 4.")
```

Run: `python test_gemini.py` — if you see a response, your API key is working.

4.4 Model Name Reference


```
# Use this exact model name throughout Phases 2-5
MODEL_NAME = "gemini-2.0-flash"

# For embeddings (Phase 2+)
EMBEDDING_MODEL = "models/text-embedding-004"
```

Section 5: LangSmith Setup

5.1 Creating a Free Account

1. Go to smith.langchain.com
2. Sign up with your personal email
3. Verify your email
4. Go to **Settings** → **API Keys**
5. Click **"Create API Key"**
6. Copy the key — it starts with `lsv2_pt_...`

5.2 Environment Variables

Add to your `.env`:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=lsv2_pt_your_key_here
LANGCHAIN_PROJECT=AI-Readiness-POC-01-P2
```

Update `LANGCHAIN_PROJECT` for each phase:

- Phase 2: `AI-Readiness-POC-01-P2`
- Phase 3: `AI-Readiness-POC-01-P3`
- Phase 4: `AI-Readiness-POC-01-P4`
- Phase 5: `AI-Readiness-POC-01-P5`

5.3 Verifying Traces

After setting env vars, run any LangChain code:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
load_dotenv()

llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash")
result = llm.invoke("Test trace")
print(result.content)
```

Go to smith.langchain.com → Your project → You should see a trace appear within seconds.

5.4 Adding Custom Metadata to Traces

```
from langsmith import traceable

@traceable(name="rag_query", metadata={"poc_id": "POC-01", "phase": 2})
def rag_query(question: str) -> str:
    # your RAG logic here
    pass
```

Section 6: ChromaDB Setup

6.1 Installation

```
pip install chromadb==0.5.3
```

6.2 Persistent Client Setup

```
import chromadb

# Persistent storage – data survives between runs
client = chromadb.PersistentClient(path="./chroma_db")

# Create or get a collection
collection = client.get_or_create_collection(
    name="poc_01_manual",
    metadata={"hnsw:space": "cosine"} # Use cosine similarity
)

print(f"Collection has {collection.count()} documents")
```

6.3 Quick Test

```
# test_chromadb.py
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="./test_chroma")
collection = client.get_or_create_collection("test")

# Add a document
collection.add(
    documents=["The loan application process takes 5-7 business days."],
    ids=["doc1"]
)
```

```
# Query
results = collection.query(
    query_texts=["How long does loan approval take?"],
    n_results=1
)
print(results['documents']) # Should return the document above
```

6.4 With LangChain

```
from langchain_chroma import Chroma
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

# Create/load vector store
vectorstore = Chroma(
    collection_name="poc_manual",
    embedding_function=embeddings,
    persist_directory="./chroma_db"
)

# Add documents
vectorstore.add_texts(["Your text chunks here"])

# Similarity search
docs = vectorstore.similarity_search("your query", k=4)
```

Section 7: GitHub Copilot Vibe Coding Guide

GitHub Copilot is pre-provisioned on your organization laptop. If not, raise a service request at <https://wiprocio.service-now.com/sp> for **Wc4c license**. This guide explains how to use it effectively for Phase 1.

7.1 Core Copilot Features

Feature	How to Use	Best For
Inline Suggestions	Type code — Copilot suggests completions	Function bodies, boilerplate
Accept Suggestion	Press Tab	Accept the full suggestion
Next Suggestion	Alt+]	Cycle through alternatives
Copilot Chat	Ctrl+I or sidebar chat	Explaining, debugging, refactoring
Slash Commands	/explain , /fix , /tests , /doc	Specific tasks in chat

7.2 Vibe Coding Workflow for Phase 1

The Vibe Coding Loop:

1. DESCRIBE → Write a comment or docstring explaining what you want
2. GENERATE → Let Copilot suggest the implementation
3. REVIEW → Read the suggestion carefully – don't blindly accept
4. REFINE → Accept, modify, or reject and re-describe more clearly
5. TEST → Verify the generated code works before moving on

Example — Generating a FastAPI endpoint:

```
# Create a POST endpoint that accepts a loan application from the request body,
# validates the applicant exists, sets status to 'submitted',
# saves to database, and returns the created application with 201 status code
@router.post("/applications", response_model=ApplicationResponse, status_code=201)
async def create_application(
    # Copilot will suggest the rest...
```

7.3 Tips for Better Suggestions

1. **Write detailed comments first** — The more context you give, the better the suggestions
2. **Use type hints** — Copilot uses types to generate better code
3. **Show a pattern once** — Write one endpoint fully, Copilot will follow the pattern for the rest
4. **Use Copilot Chat for models** — Ask "Generate a SQLAlchemy model for a LoanApplication with these fields: ..."
5. **Use `/tests` command** — Select a function and ask Copilot to write tests for it

7.4 Phase 1 Copilot Checklist

Associates must use Copilot for at least 60% of Phase 1 code. Document usage by keeping a simple log:

```
## Copilot Usage Log
| File | Function/Component | Used Copilot? | Notes |
|-----|-----|-----|-----|
| models.py | LoanApplication model | Yes | Generated full model from comment |
| routers/applications.py | create_application | Yes | Needed small edits |
| components/ApplicationForm.jsx | Form component | Yes | Mostly accepted as-is |
```

Section 8: OpenTelemetry Setup (Python)

8.1 Basic Setup

```
# otel_setup.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
```

```
from opentelemetry.sdk.trace.export import ConsoleSpanExporter, BatchSpanProcessor
from opentelemetry.sdk.resources import Resource

def setup_telemetry(service_name: str, poc_id: str, phase: int):
    resource = Resource.create({
        "service.name": service_name,
        "poc.id": poc_id,
        "poc.phase": str(phase),
    })

    provider = TracerProvider(resource=resource)
    # Console exporter for local dev
    provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
    trace.set_tracer_provider(provider)

    return trace.get_tracer(service_name)

# Usage
tracer = setup_telemetry("loan-app-api", "POC-01", 1)
```

8.2 Creating Spans

```
# Manual span creation
with tracer.start_as_current_span("create_application") as span:
    span.set_attribute("application.loan_type", loan_type)
    span.set_attribute("application.amount", amount)
    try:
        result = create_application_in_db(...)
        span.set_attribute("application.id", result.id)
        span.set_status(trace.StatusCode.OK)
        return result
    except Exception as e:
        span.record_exception(e)
        span.set_status(trace.StatusCode.ERROR, str(e))
        raise
```

8.3 FastAPI Auto-Instrumentation

```
# In main.py
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

app = FastAPI()
FastAPIInstrumentor.instrument_app(app)
# All HTTP requests now automatically create spans
```

8.4 Required Span Names Per Phase

Phase	Required Span Names
Phase 1	<code>http.request</code> , <code>db.query</code> , <code>auth.validate</code>
Phase 2	<code>rag.document_load</code> , <code>rag.chunk</code> , <code>rag.embed</code> , <code>rag.retrieve</code> , <code>rag.generate</code>
Phase 3	<code>agent.tool_call</code> , <code>agent.reasoning</code> , <code>api.call</code> , <code>agent.summarize</code>
Phase 4	<code>mcp.tool_invoke</code> , <code>chat.message</code> , <code>mcp.response</code>
Phase 5	<code>agent.{name}.activate</code> , <code>supervisor.route</code> , <code>graph.execute</code>

Section 9: Frequently Asked Questions

- Q: I get "API quota exceeded" from Gemini — what do I do?** A: The free tier allows 15 requests/minute. Add `time.sleep(4)` between LLM calls in batch operations. For testing, use a mock LLM.
- Q: ChromaDB gives a "dimension mismatch" error — what happened?** A: You created the collection with one embedding model and are now using a different one. Delete the `./chroma_db` folder and re-ingest.
- Q: LangSmith traces aren't appearing — why?** A: Check that `LANGCHAIN_TRACING_V2=true` is set and that your `LANGCHAIN_API_KEY` is correct. Make sure you're calling `load_dotenv()` before any LangChain imports.
- Q: My SQLite database is locked — how do I fix it?** A: Close the SQLite Viewer in VS Code, or any DB browser that has the file open. SQLite only allows one writer at a time.
- Q: Can I use a different LLM instead of Gemini?** A: No — Gemini 2.0 Flash via Google AI Studio is the standardized LLM for this program to ensure consistent free-tier access. Using other LLMs may affect test case scoring.